OCR H446 / PAPER 1 / UNIT 1.1

Processors, input/output and storage

CHAPTER CONTENTS

Select a topic to jump to its original worked pages.

1.1.1(a,b) / CPU registers and FDE	4
1.1.1(c,d) / Performance and pipelining	8
1.1.1(e) / CPU architectures	12
1.1.2(a) / RISC and CISC	16
1.1.2(b,c) / GPUs and parallel systems	20
1.1.3(a,b) / Device selection and storage	24
1.1.3(c,d) / Memory and virtual storage	28

H446/01 / REVISION

Understand it. Model it. Try it. Check it.

Use this chapter to understand the theory, follow worked models and practise independently.

VISUAL ROUTE THROUGH SECTION 1.1

1.1.1
Processor function, throughput and architectures

1.1.2
RISC/CISC, GPUs and parallel systems

1.1.3
Choosing devices, memory and virtual storage

ONE CONSISTENT REVISION ROUTINE

01 UNDERSTAND
A visual model connected to precise A Level terms.

02 MODEL
A complete worked example with every step visible.

03 TRY
Original exam-style questions and writing room.

04 CHECK
Indicative marking points and green-pen corrections.

WHAT IS PRESERVED

Follow CPU diagrams, worked traces and exam-style practice.

Move from the visual memory hook to precise processor terminology.

1.1 / SPECIFICATION ALIGNMENT

Coverage before page count

The official OCR H446 specification identifies the requirements below. Seven four-page learning cycles cover all 12 sub-points in section 1.1.

SPEC	SPECIFICATION CONTENT	
1.1.1(a,b)	ALU, CU, PC/ACC/MAR/MDR/CIR, buses and the fetch-decode-execute cycle	pp. 3-6
1.1.1(c,d)	Clock speed, cores, cache and pipelining	pp. 7-10
1.1.1(e)	Von Neumann, Harvard and contemporary architecture	pp. 11-14
1.1.2(a)	RISC and CISC differences and uses	pp. 15-18
1.1.2(b,c)	GPUs, multicore and parallel systems	pp. 19-22
1.1.3(a,b)	Selecting I/O, magnetic, flash and optical devices	pp. 23-26
1.1.3(c,d)	RAM, ROM and virtual storage	pp. 27-30

REVISION CHECK

Explain CPU components, buses and register changes using precise terms.

Source: OCR H446 specification, Section 1.1. Questions are original RHSC revision practice.

1.1.1(a,b) / SEE IT / REGISTER ROLES

Follow the instruction, not just the arrow

A register stores a particular kind of information. Track the location, transferred value, instruction and calculated result separately.

FOUR VALUES ARE NOT INTERCHANGEABLE

PC / NEXT

Holds address of next instruction in a normal sequential trace.

MAR / ADDRESS

Holds address currently being accessed in main memory.

MDR / TRANSFER

Holds data or instruction being transferred to/from memory.

CIR / CURRENT

Holds instruction being decoded or executed.

TRACE ONE INSTRUCTION: RAM[40] = ADD 85

PC	MAR	MDR	CIR
40	40	ADD 85	ADD 85

FROM DIAGRAM TO EXAM LANGUAGE

Fetch: the PC address is copied to MAR; memory is read via MDR into CIR; PC normally advances. Decode: the CU interprets the instruction. Execute: the ALU or another unit performs the operation. ACC can retain arithmetic results.

WORDS THAT EARN MARKS

- **PC** Next instruction
- **MAR** Accessed address
- **MDR** Transferred value
- **CIR** Current instruction

VISUAL MEMORY BRIDGE / FULL-SIZE ORIGINAL ANALOGY

CPU HIGHWAY: one connected system - see PDF page 338

1.1.1(a,b) / WORKED EXAMPLE / TEACHING CPU

ADD 85: a full fetch-and-execute trace

At the start, PC = 40, ACC = 9, RAM[40] = ADD 85, RAM[85] = 6. Here the operand 85 is a memory address.

PHASE	WHAT CHANGES	WHY IT CHANGES
Fetch 1	MAR ← PC = 40	Select RAM address of instruction.
Fetch 2	MDR ← RAM[40] = ADD 85	Transfer instruction on data bus.
Fetch 3	CIR ← MDR; PC ← 41	Store current instruction; move on.
Decode	CU identifies ADD and operand 85	Determine operation and operand address.
Execute 1	MAR ← 85; MDR ← RAM[85] = 6	Read the operand, not address 85 as a value.
Execute 2	ACC ← 9 + 6 = 15	ALU performs arithmetic; ACC stores result.

END STATE

PC = 41; CIR = ADD 85; ACC = 15; MAR = 85; MDR = 6 (in this simplified teaching trace).

Where the model stops: a real CPU can have additional registers, different buses and instruction formats. A jump/branch can change the PC rather than allowing sequential advancement.

VISUAL MEMORY BRIDGE / FULL-SIZE ORIGINAL ANALOGY

CITY HALL: six-step register trace - see PDF page 340

1.1.1(a,b) / INDEPENDENT EXAM PRACTICE

Trace, label and explain

Assume a simplified accumulator CPU unless stated otherwise.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 3 MARKS

State the different roles of PC, MAR and MDR in fetching an instruction.

QUESTION 2 | 5 MARKS

PC = 72, ACC = 4, RAM[72] = ADD 90 and RAM[90] = 8. State the final PC, CIR and ACC, showing the operand read.

QUESTION 3 | 6 MARKS

Explain how the address, data and control buses contribute to fetching an instruction and then obtaining an operand.

1.1.1(a,b) / SELF-MARKING / INDICATIVE ANSWERS

Check each register transfer

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 3 MARKS

- PC holds the address of the next instruction [1].
- MAR holds the memory location being accessed [1].
- MDR holds data or instruction transferred to/from memory [1].

Q2 / 5 MARKS

- MAR receives 72 [1]; RAM[72] supplies ADD 90 through MDR [1].
- CIR becomes ADD 90 [1]; PC normally advances to 73 [1].
- CPU reads RAM[90] = 8 and ACC becomes 4 + 8 = 12 [1].

Q3 / 6 MARKS

- Address bus selects a memory location [1]; control bus signals READ [1].
- Data bus carries fetched instruction towards CPU [1]; MDR/CIR retain it [1].
- An operand memory address is subsequently placed on address bus [1]; data bus returns operand for ALU operation [1].

COMMON MISCONCEPTION

ADD 90 means add the value stored at memory address 90. It does not necessarily mean add the literal number 90.

Green pen: Choose the least precise of your bus or register explanations and rewrite it as a full cause-and-effect sentence.

VISUAL MEMORY BRIDGE / FULL-SIZE ORIGINAL ANALOGY

THREE HIGHWAYS: address / data / control - see PDF page 339

1.1.1(c,d) / SEE IT / BOTTLENECKS

A faster clock is not the whole story

Speed depends on the workload and implementation. Clock frequency, cache access, core count and pipeline efficiency influence different bottlenecks.

THREE PROCESSOR LEVERS: THREE DIFFERENT QUESTIONS

CLOCK SPEED

How many clock cycles occur each second? Work per cycle still matters.

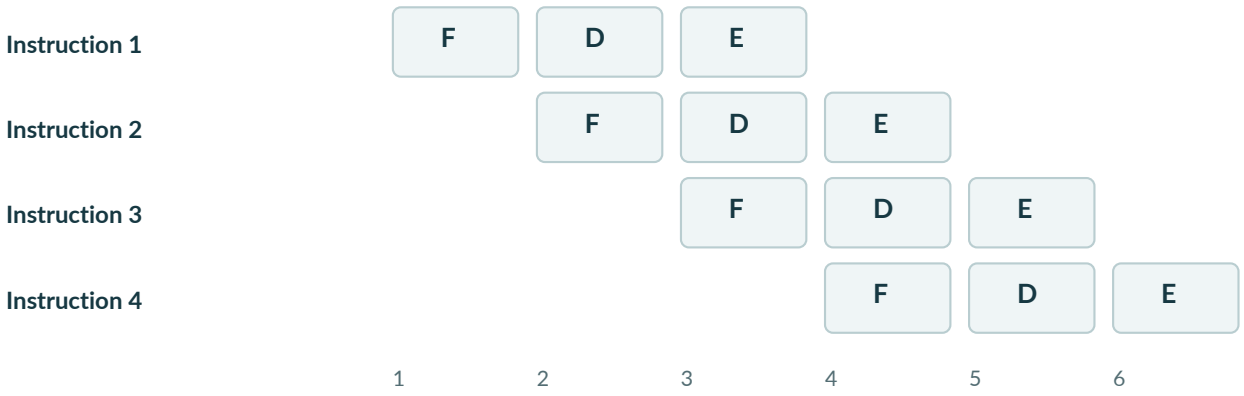
CACHE

Can frequently used instructions/data avoid slower main-memory access?

NUMBER OF CORES

Can multiple independent threads/tasks actually run in parallel?

PIPELINE: OVERLAP INSTRUCTION STAGES



One simplified cycle per stage: after the pipeline fills, a result may finish each cycle, provided there are no stalls. The latency of a single instruction need not become shorter.

WHERE THE ANALOGY STOPS

Data dependencies, branches and shared resources can stall or flush work. Real pipelines have different stage timings and overhead; ideal timing is a model, not a guarantee.

WORDS THAT EARN MARKS

- **Clock** Cycles/sec
- **Cache** Fast nearby data
- **Pipeline** Overlap stages
- **Stall** Delay a stage

1.1.1(c,d) / WORKED EXAMPLE / THREE STAGES

Compare serial and pipelined execution

Four independent instructions use fetch (2 ns), decode (2 ns) and execute (2 ns). Assume an ideal pipeline, no overhead or hazards.

WITHOUT OVERLAP / 4 INSTRUCTIONS

INSTR. 1	INSTR. 2	INSTR. 3	INSTR. 4
F 2 + D 2 + E 2	F 2 + D 2 + E 2	F 2 + D 2 + E 2	F 2 + D 2 + E 2

4 × (2 + 2 + 2) = **24 ns** total

WITH OVERLAP / COMPLETE A NEW INSTRUCTION EVERY 2 NS (IDEAL)

INTERVAL	STAGE F	STAGE D	STAGE E
0-2 ns	Instruction 1	-	-
2-4 ns	Instruction 2	Instruction 1	-
4-6 ns	Instruction 3	Instruction 2	Instruction 1
6-8 ns	Instruction 4	Instruction 3	Instruction 2
8-10 ns	-	Instruction 4	Instruction 3
10-12 ns	-	-	Instruction 4

EXPLAIN THE RESULT

The first instruction still takes 6 ns; total time is 6 + 3 × 2 = **12 ns**. Under these ideal assumptions the four-instruction batch completes in half the non-pipelined time.

Important: this example teaches increased **throughput**, not a shorter single-instruction **latency**. Actual speed-up can be lower if dependencies or branches introduce stalls.

1.1.1(c,d) / INDEPENDENT EXAM PRACTICE

Prove, calculate, evaluate

Show the units and state the ideal assumptions in numerical answers.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 3 MARKS

Explain why a CPU with a higher clock frequency might not always complete a program sooner.

QUESTION 2 | 5 MARKS

Five independent instructions each require F, D and E stages of 2 ns. Calculate the non-pipelined and ideal pipelined completion times, then the speed-up.

QUESTION 3 | 6 MARKS

Evaluate how cache, more cores and pipelining could affect a CPU running a program with some branches and limited parallel work.

1.1.1(c,d) / SELF-MARKING / INDICATIVE ANSWERS

What improves speed, and when?

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 3 MARKS

- Clock speed is cycles per second [1].
- Different CPUs can do different work per cycle [1].
- Cache misses, instruction mix or stalls may outweigh frequency advantage [1].

Q2 / 5 MARKS

- One instruction uses 3 × 2 = 6 ns [1].
- Non-pipelined five-instruction time = 5 × 6 = 30 ns [1].
- Ideal pipelined time = 6 + 4 × 2 = 14 ns [1].
- Speed-up = 30 / 14 ≈ 2.14× [1]; no stalls/overhead assumed [1].

Q3 / 6 MARKS

- Cache can reduce average main-memory access time on hits [1]; effectiveness depends on locality/hit rate [1].
- Multiple cores can run independent threads concurrently [1], but sequential code cannot benefit in the same way [1].
- Pipelining overlaps stages to raise throughput [1]; branches/data dependencies may stall or flush it [1].

COMMON MISCONCEPTION

Pipelining improves throughput in a well-filled pipeline. It does not simply divide the latency of one instruction by the stage count.

Green pen: Show your 14 ns calculation as a timeline rather than only writing a speed-up number.

1.1.1(e) / SEE IT / ARCHITECTURES

Two routes, one critical difference

Von Neumann and Harvard describe memory organisation. Contemporary processors can combine ideas at different levels of the memory hierarchy.

MEMORY ACCESS PATHS

VON NEUMANN

One shared memory / pathway
Instructions and data share an address space and access path in the simplified model.

Potential limit: instruction fetch and data access can contend.

HARVARD

Separated instruction and data stores/paths
Fetching instructions and accessing data can use separate routes.

Potential benefit: concurrent access to the two.

A CONTEMPORARY DESIGN CAN BE MIXED

INSTRUCTION CACHE

Dedicated nearby instruction path.

DATA CACHE

Dedicated nearby data path.

SHARED LOWER-LEVEL MEMORY

Eventually accesses shared wider memory.

Separate L1 instruction/data caches can resemble Harvard close to the CPU while lower-level memory still uses a unified address space. Architecture labels describe organisation, **not a guaranteed performance ranking**.

EXAM LANGUAGE

Shared instruction/data route can create a Von Neumann bottleneck. Separate routes can allow simultaneous accesses. Actual performance also depends on cache, bandwidth, hazards and implementation.

VISUAL MEMORY BRIDGE / FULL-SIZE ORIGINAL ANALOGY

TWO ROADS: Von Neumann vs Harvard - see PDF page 341

1.1.1(e) / WORKED EXAMPLE / ACCESS DEMAND

Predict a memory-access bottleneck

A simplified CPU needs an instruction fetch and one data read during the same clock interval.

SCENARIO A / SINGLE SHARED MEMORY ACCESS PORT

CLOCK SLOT	REQUEST	OUTCOME
Slot 1	Fetch next instruction	One use of the shared port.
Slot 2	Read operand	Must use the port later if both conflict.

SCENARIO B / SEPARATE PATHS AVAILABLE

CLOCK SLOT	INSTRUCTION PATH	DATA PATH
Slot 1	Fetch next instruction	Read operand at the same time

SO WHAT?

For this simplified pair of memory operations, separate paths avoid the shared-port conflict. This alone does **not** prove the whole program finishes twice as quickly.

TURN OBSERVATION INTO A COMPARISON

Weak: “Harvard is faster.”
Stronger: “Separate instruction and data pathways may allow concurrent fetch and data access, reducing contention where both are needed; other bottlenecks can remain.”

1.1.1(e) / INDEPENDENT EXAM PRACTICE

Compare architectures precisely

Avoid unsupported claims about guaranteed speed.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 3 MARKS

State three structural differences or similarities between simplified Von Neumann and Harvard architectures.

QUESTION 2 | 5 MARKS

A processor often reads data while fetching upcoming instructions. Explain why a shared memory pathway may reduce throughput and how separate pathways help.

QUESTION 3 | 6 MARKS

A modern CPU has separate instruction and data L1 caches but uses shared main memory. Explain why this is a hybrid arrangement and evaluate the claim that it eliminates memory bottlenecks.

1.1.1(e) / SELF-MARKING / INDICATIVE ANSWERS

Use architecture vocabulary

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 3 MARKS

- Von Neumann model uses a shared instruction/data memory space [1].
- The access path may be shared between fetching and data transfers [1].
- Harvard uses distinct instruction and data storage/access paths [1].

Q2 / 5 MARKS

- Both instruction fetch and data read demand memory access [1].
- Shared port/path cannot serve conflicting demands simultaneously [1].
- One request may wait, reducing available throughput [1].
- Separate instruction/data paths can permit the two accesses in parallel [1].
- Benefit is conditional on implementation and workload [1].

Q3 / 6 MARKS

- Separate L1 caches provide distinct nearby instruction/data paths [1].
- Main memory still uses a shared system/address space [1].
- Hence it combines Harvard-like cache separation with a unified wider memory model [1].
- A cache miss can still access slower shared memory [1].
- Bandwidth/latency and contention remain relevant [1].
- Therefore it can reduce some conflicts but cannot eliminate all bottlenecks [1].

COMMON MISCONCEPTION

Separate caches alone do not mean that the whole machine has fully separate instruction and data address spaces.

Green pen: Rewrite any unqualified “always faster” statement as a conditional explanation.

1.1.2(a) / SEE IT / TWO INSTRUCTION-SET IDEAS

Instruction count is not execution time

RISC and CISC are instruction-set design approaches. Both can be implemented in sophisticated modern processors.

ONE CALCULATION, TWO ILLUSTRATIVE ENCODINGS

RISC-LIKE SEQUENCE

```
LOAD R1, [A]
LOAD R2, [B]
ADD R3, R1, R2
STORE [C], R3
```

The operations are simpler and load/store-style. This is teaching pseudocode, not a particular ISA.

CISC-LIKE EXAMPLE

```
ADD_MEMORY C, A, B
```

One **hypothetical** compound instruction expresses more work; decoding and execution may still involve several internal steps.

COMPARE CHARACTERISTICS WITHOUT CLAIMING ABSOLUTES

RISC / TYPICAL	CISC / TYPICAL
More regular, simpler instructions	Richer, sometimes variable-length instructions
Load/store style is common	More addressing modes may be available
Decoding and pipelining can be simpler	Complex instructions may improve code density
May need more instructions for same job	Fewer instructions does not imply fewer cycles

WHY THIS MATTERS

Performance depends on implementation, clock rate, caches, compiler and workload. Instruction-set category alone is not a reliable speed comparison.

VISUAL MEMORY BRIDGE / FULL-SIZE ORIGINAL ANALOGY

RISC / CISC: different routes - see PDF page 342

1.1.2(a) / WORKED EXAMPLE / NO MADE-UP TIMINGS

Compare the cost of two sequences

Assume A holds 3 and B holds 5. Both illustrative sequences compute C = A + B.

STEP	LOAD / STORE APPROACH	COMPOUND APPROACH
1	LOAD R1, [A] -> R1 = 3	ADD_MEMORY C, A, B
2	LOAD R2, [B] -> R2 = 5	Meaning: read A and B; compute 8; write C
3	ADD R3, R1, R2 -> R3 = 8	Implementation and timing unspecified
4	STORE [C], R3 -> C = 8	Possible internal micro-operations

MODEL ANSWER: WHAT CAN WE CONCLUDE?

Both approaches produce C = 8. The compound form uses fewer **visible instructions**. We cannot claim it runs faster: each instruction may take different cycles and incur different memory/decode work.

HOW TO STRUCTURE A SIX-MARK COMPARISON

CHARACTERISE

Discuss typical instruction complexity and code density.

EXPLAIN TRADE-OFF

Link decoding / pipelines / memory accesses to possible benefits or costs.

EVALUATE

Conclude performance depends on actual hardware and task.

1.1.2(a) / INDEPENDENT EXAM PRACTICE

Compare instead of repeating slogans

Use typical characteristics and explain why exceptions exist.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 3 MARKS

State one characteristic of RISC, one of CISC, and the meaning of code density.

QUESTION 2 | 5 MARKS

Compare LOAD/LOAD/ADD/STORE with one hypothetical complex instruction to calculate C = A + B. Explain why fewer instructions need not mean faster execution.

QUESTION 3 | 6 MARKS

Evaluate “RISC processors are always faster than CISC processors” with reference to decoding, pipelining, instruction count and memory accesses.

1.1.2(a) / SELF-MARKING / INDICATIVE ANSWERS

Distinguish ISA from implementation

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 3 MARKS

- RISC typically uses simpler/more regular instructions [1].
- CISC can provide richer/compound operations [1].
- Code density is functionality encoded per unit of machine-code size [1].

Q2 / 5 MARKS

- LOAD reads A [1] and another LOAD reads B [1].
- ADD combines register values [1]; STORE writes result C [1].
- Compound instructions can have more internal operations/cycles; counting instructions alone does not establish execution time [1].

Q3 / 6 MARKS

- RISC regularity can simplify decoding [1] and help efficient pipelining [1].
- But a task may require more RISC instructions, creating fetch/code-size costs [1].
- CISC can express some work with fewer instructions / greater code density [1].
- CISC decoding may be complex, though implementation can mitigate it [1].
- Caches, compiler, microarchitecture and workload determine actual speed; the absolute claim is false [1].

COMMON MISCONCEPTION

RISC does not mean that every instruction completes in one cycle; CISC does not mean a complex instruction is automatically slower.

Green pen: Compare instruction count, cycles per instruction and workload.

1.1.2(b,c) / SEE IT / DATA PARALLELISM

Match the hardware to the work

A CPU can combine relatively few powerful cores with flexible control flow. GPUs offer many execution units suited to repeated work on large data sets.

DIFFERENT SHAPE OF TASKS

MULTICORE CPU

Core 1 Core 2 Core 3 Core 4

Different threads can make progress in parallel, but shared data may need synchronisation.

GPU WORKLOAD

Many image pixels can receive similar calculations concurrently. Branch-heavy, tiny sequential tasks may use GPU resources poorly.

A VISUAL OF 16 INDEPENDENT ITEMS

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16

For a data-parallel operation, items 1–16 can be divided among execution units. The **serial part** (loading, control, final aggregation) still costs time, and communication may add overhead.

WHERE IT IS USEFUL

GPU applications extend beyond drawing graphics: machine-learning tensor operations, scientific simulations and parallel image processing can use many similar independent calculations.

WORDS THAT EARN MARKS

- **Core** CPU executor
- **Thread** Execution flow
- **GPU** Data parallel
- **Sync** Coordinate tasks

1.1.2(b,c) / WORKED EXAMPLE / SERIAL FRACTION

Why four cores do not give 4× speed

A job takes 16 time units on one core: 4 units are inherently sequential and 12 can be divided perfectly over four cores.

ONE CORE / BASELINE



FOUR CORES / BEST-CASE CALCULATION

First: serial stage = 4 units

THEN / FOUR CORES WORK SIMULTANEOUSLY

- Core 1: 3 units
- Core 2: 3 units
- Core 3: 3 units
- Core 4: 3 units

WORKED CALCULATION

Sequential 4 + parallel 12 ÷ 4 = **7 units**. Ideal speed-up = 16 ÷ 7 ≈ **2.29×**, not 4×. Synchronisation, memory bandwidth or imbalance could make real results worse.

Transfer this reasoning: more cores or a GPU only accelerate work they can execute concurrently. Always identify serial work before predicting speed-up.

1.1.2(b,c) / INDEPENDENT EXAM PRACTICE

Apply parallel thinking

Show how a task is divided and any unavoidable sequential stage.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 3 MARKS

Give one graphics and one non-graphics GPU application and explain one feature that makes both suitable.

QUESTION 2 | 5 MARKS

A job takes 16 units on one core: 4 are serial and 12 can be split over four cores. Calculate the ideal four-core completion time and speed-up, and identify one reason reality may be slower.

QUESTION 3 | 6 MARKS

Evaluate whether a GPU or a multicore CPU is the better choice for a workload that transforms millions of pixels but also performs some sequential file handling.

1.1.2(b,c) / SELF-MARKING / INDICATIVE ANSWERS

Find the serial limit

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 3 MARKS

- Graphics example: applying an image filter/rendering [1].
- Non-graphics example: matrix/tensor computations in machine learning [1].
- Many data elements use similar independent operations [1].

Q2 / 5 MARKS

- Serial portion remains 4 units [1].
- Parallel portion per core is 12/4 = 3 units [1].
- Total ideal time is 4 + 3 = 7 units [1].
- Speed-up is 16/7 ≈ 2.29× [1].
- Synchronization/data-transfer/memory contention/imbalance can reduce gain [1].

Q3 / 6 MARKS

- Many pixel transformations can be performed independently [1]; GPU excels at this data-parallel section [1].
- Sequential file handling may suit a CPU core [1]; it does not scale across GPU units in the same way [1].
- Transfers to/from GPU or memory bandwidth may cost time [1].
- A hybrid CPU/GPU design is justified; overall speed depends on the serial portion and overhead [1].

COMMON MISCONCEPTION

The maximum speed-up is not the number of cores when serial code remains. GPU and multicore CPU performance both depend on task structure.

Green pen: Circle each claim about speed and add the relevant assumption or limit.

1.1.3(a,b) / SEE IT / PRACTICAL CONSTRAINTS

Choose by the problem, not the product

A strong H446 answer names a suitable device and links a property of that device directly to the scenario.

THE SCENARIO DECIDES THE CRITERIA

INPUT

Barcode reader: fast, consistent item capture at a stockroom checkout.

OUTPUT

E-ink screen: low power for infrequently changing text, not rapid animation.

STORAGE

SSD: fast random access and no moving read/write mechanism for a moving device.

COMPARE STORAGE MEDIA USING NEEDS

MEDIUM	ADVANTAGE	LIMIT / TYPICAL MATCH
Magnetic disk / tape	Often economical for large capacity	Disk has moving parts; tape sequential; suitable bulk archives
Flash / SSD	Fast access, resistant to movement	Cost/GB can be higher; suitable active portable work
Optical disc	Physical, inexpensive distribution of a fixed copy	Capacity/speed limits; suitable offline distribution

EXAM METHOD / CHOOSE -> PROPERTY -> BECAUSE

"Use flash storage **because** the data logger moves over rough ground and the device has no moving read/write head." This earns more than "flash is good."

1.1.3(a,b) / WORKED EXAMPLE / THREE SCENARIOS

Make a defensible recommendation

A wildlife monitoring project has moving data loggers; an office has live shared files; a school archives infrequently accessed historical footage.

NEED	CHOICE	REASON LINKED TO NEED
Moving logger	Flash storage	No moving read/write parts; more tolerant of vibration.
Active office work	SSD in server	Fast random access suits repeated small-file reads and writes.
Large cold archive	Magnetic tape	High capacity and economical archival media; sequential access acceptable.

NOW ASSESS A COMPETING OPTION

WHY NOT OPTICAL?

Optical disc media can distribute an offline fixed copy, but may be inconvenient for large, frequently updated, high-volume archives.

WHY NOT SSD FOR EVERYTHING?

SSD is responsive, but the cheaper cost per capacity of an alternative may matter when retrieval is rare and large volumes dominate.

EVALUATIVE CONCLUSION

Select by capacity, cost, speed, portability, durability, access pattern and maintenance. No medium is universally best; the priorities decide.

1.1.3(a,b) / INDEPENDENT EXAM PRACTICE

Justify each technology choice

State the device, then give a property and a consequence in the scenario.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 3 MARKS

A museum needs visitors to hear recorded descriptions at exhibits, while preventing the sound disturbing nearby areas. Recommend an output device and explain two reasons.

QUESTION 2 | 5 MARKS

A school needs fast storage for working files and low-cost storage for a rarely accessed video archive. Recommend a medium for each and explain why it fits.

QUESTION 3 | 6 MARKS

Evaluate magnetic, flash and optical storage for a portable field recorder that is frequently carried, must write data rapidly and occasionally shares completed recordings offline.

1.1.3(a,b) / SELF-MARKING / INDICATIVE ANSWERS

Select, justify, qualify

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 3 MARKS

- Headphones/earphones are suitable [1].
- Direct sound to the visitor without broadcasting to the room [1].
- Other visitors are less disturbed; suitable for simultaneous nearby exhibits [1].

Q2 / 5 MARKS

- Flash SSD is appropriate for active working files [1].
- It provides responsive random access for many reads/writes [1].
- Magnetic tape or suitably priced magnetic storage can serve a cold archive [1].
- It offers economical large capacity [1]; slower/sequential retrieval is acceptable because files are rarely requested [1].

Q3 / 6 MARKS

- Flash has no moving read/write parts and tolerates carrying/vibration [1].
- It typically provides fast writes/access for regular recording [1].
- Magnetic disk may offer economical capacity but mechanical vulnerability in a field device [1].
- Optical media can provide removable offline copies [1].
- Optical capacity/write speed/rewrite convenience may limit primary recording use [1].
- A justified combination is flash for recording plus appropriate removable/export format for distribution [1].

COMMON MISCONCEPTION

“Magnetic is slow” is too vague: hard disks and tape have different access patterns. Explain the medium and the specific workload.

Green pen: Underline each “because” in your answers. Add one wherever a choice has no scenario-specific justification.

1.1.3(c,d) / SEE IT / MEMORY VERSUS STORAGE

What disappears when power is off?

RAM, ROM and virtual storage solve different problems. In this specification point, virtual storage is not virtual memory.

THREE DIFFERENT FUNCTIONS

RAM

Volatile working memory for instructions and data in active use; normally read/write.

ROM / FIRMWARE

Non-volatile startup / embedded instructions; contents remain without power.

VIRTUAL STORAGE

Remote/network-backed files presented as a convenient logical store.

WHERE THE FILES REALLY LIVE

USER DEVICE

Opens a familiar drive/folder.

›

NETWORK

Carries requests and returned data.

›

REMOTE STORE

Physical disks/SSDs hold files.

CRITICAL DISTINCTION

Virtual storage (1.1.3d) makes remote or pooled secondary storage appear accessible as a logical drive. **Virtual memory (1.2.1b)** uses secondary storage to support a process address space when RAM is constrained.

Access and backup: remote access can help sharing and off-site recovery, but synchronised deletion can propagate. Separate retained backups and tested restoration are still needed.

WORDS THAT EARN MARKS

RAM Volatile working

ROM Non-volatile code

Virtual Remote/logical

Backup Recover earlier

1.1.3(c,d) / WORKED EXAMPLE / JUSTIFY AND WARN

Choose storage for two offices

A design team has offices in two towns. Staff need to work on shared files, and storage requirements are expected to grow.

STEP	OBSERVATION	EXPLANATION
1	Files appear on a shared drive	The interface is logical; data lives on remote servers.
2	Both offices use the same project store	Network access makes one managed set of files available.
3	The provider expands capacity	Physical disks/SSDs can be added without changing user paths.
4	A link or service fails	Uncached remote files may be temporarily unavailable.
5	A deletion synchronises	Independent backup/version retention is required to recover.

TURN TO EXAM WORDING

Virtual storage is useful here because staff in separate sites can access centrally held files and the logical capacity can expand. It depends on the network/provider, and the data must still be stored physically somewhere.

DO NOT CONFUSE THE TERMS

RAM: fast, volatile working store. **ROM:** non-volatile startup/firmware. **Remote storage:** files behind a logical drive. **Virtual memory:** address-space management using secondary storage, covered separately under 1.2.1.

1.1.3(c,d) / INDEPENDENT EXAM PRACTICE

Tell memory and storage apart

The final question requires both benefit and limitation.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 3 MARKS

Explain one role and one property of RAM, and contrast this with one role of ROM in a computer.

QUESTION 2 | 5 MARKS

Explain how a shared network drive can be virtual storage and distinguish it from virtual memory.

QUESTION 3 | 6 MARKS

A school stores staff files in a synchronised cloud drive. Evaluate two benefits and two risks, and explain why an independent backup is still needed.

1.1.3(c,d) / SELF-MARKING / INDICATIVE ANSWERS

Remote files are still physical

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 3 MARKS

- RAM holds instructions/data in active use [1].
- RAM is volatile (normally read/write) [1].
- ROM is non-volatile and can hold startup/firmware code [1].

Q2 / 5 MARKS

- Virtual storage presents a logical drive/folder to the user [1].
- Files are physically kept on network/remote/cloud storage [1].
- It requires network access for uncached files [1].
- Virtual memory uses secondary storage as part of address-space/RAM management [1].
- The two uses of “virtual” refer to different abstractions [1].

Q3 / 6 MARKS

- Remote files can be accessed from different locations [1]; capacity can expand centrally [1].
- Network/provider unavailability can block access [1]; third-party security/privacy and bandwidth are risks [1].
- Synchronization can propagate accidental deletion/corruption [1].
- Independent versioned backups and restore testing enable recovery of prior states [1].

COMMON MISCONCEPTION

Virtual storage does not eliminate physical storage. A synchronised cloud folder is not automatically a reliable, independent backup.

Green pen: Correct any use of “virtual memory” when you actually mean remote/cloud storage.